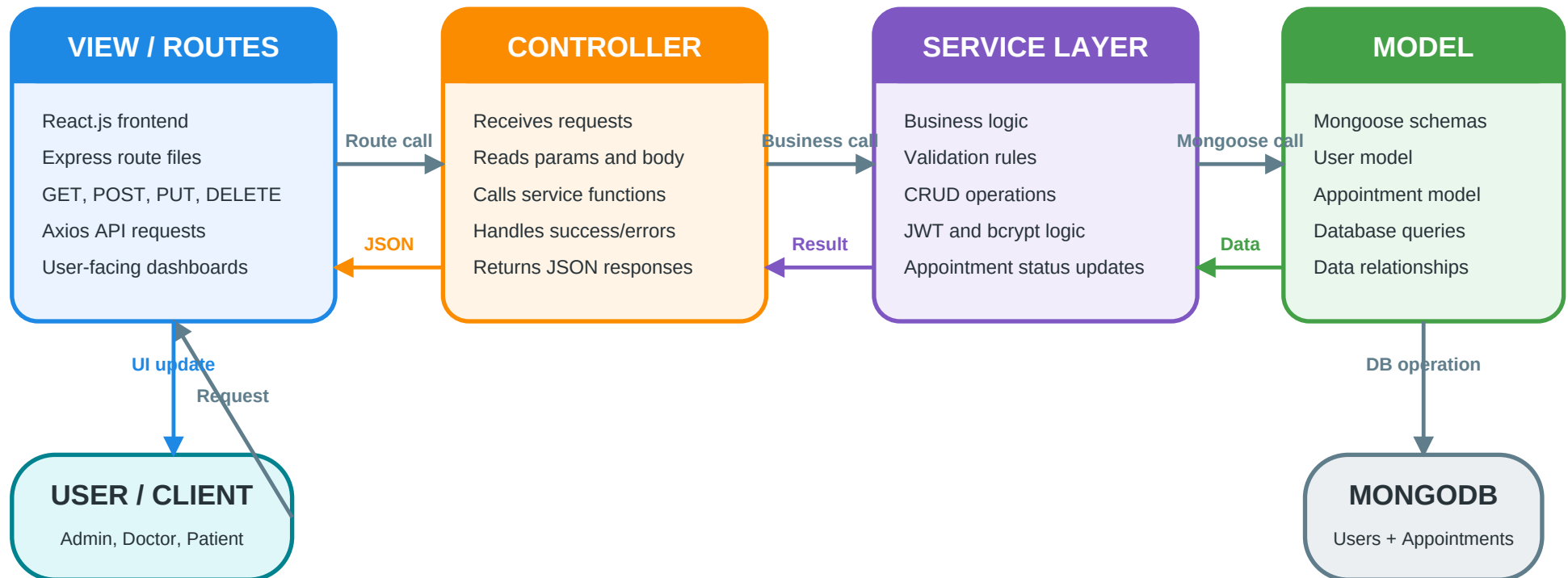


BOOK A DOCTOR SYSTEM - MVC PATTERN

Model-View-Controller architecture with an additional Service Layer

REQUEST AND RESPONSE FLOW



MVC Pattern Description

The Book a Doctor application follows the **Model-View-Controller (MVC)** architectural pattern. The backend also includes a dedicated **Service Layer** between controllers and models. This separation improves modularity, maintainability, testability, and scalability.

Model Layer (Data Layer)

The Model layer handles all data-related logic. It defines MongoDB schemas using Mongoose and performs database operations. In this project, the main models are **User** and **Appointment**. The User model stores Admin, Doctor, and Patient details, while the Appointment model stores doctor and patient references, appointment date, time, reason, and status.

Controller Layer

The Controller layer receives incoming requests from routes, reads request parameters and body data, invokes the correct service function, handles errors, and returns JSON responses. Examples include authentication, doctor, appointment, admin, and profile controllers.

View Layer (Frontend and Routing Layer)

The user-facing View is implemented using React.js pages and components. In the backend, the routing layer defines REST API endpoints such as GET, POST, PUT, and DELETE and maps each endpoint to the appropriate controller function.

Service Layer

The Service Layer contains the main business logic of the application. It performs user registration, password hashing, login validation, JWT generation, doctor management, appointment booking, appointment status updates, and profile operations. This keeps the controllers small and reusable.

Database Layer

MongoDB is used as the database, and Mongoose provides schema definition, validation, queries, and relationships. Appointment documents reference both patient and doctor users using ObjectId fields.

Request Flow in This Project

Step	Flow	Example
1	User sends a request from the React interface.	Patient clicks Book Appointment.
2	Axios sends the HTTP request to an Express route.	POST /api/appointments

Step	Flow	Example
3	JWT middleware authenticates the user.	Bearer token is verified.
4	The route calls the controller.	createAppointment controller
5	The controller calls the service layer.	bookAppointment service
6	The service uses the Mongoose model.	Appointment.create()
7	MongoDB stores or retrieves the data.	Appointment document saved.
8	A JSON response returns to the frontend.	Booking success and appointment data.

Advantages of Using MVC

- **Separation of Concerns:** Every layer has a clearly defined responsibility.
- **Scalability:** New modules can be added by creating new routes, controllers, services, and models.
- **Reusability:** Service and model logic can be reused across multiple controllers.
- **Testing:** Individual layers can be tested independently.
- **Maintainability:** Bugs and updates can be handled without changing the complete application.
- **Collaboration:** Team members can work on frontend, controllers, services, and models separately.